

Sistemas Livres: Guia Prático de Infraestrutura Linux com Debian e OpenWRT

Um laboratório pessoal de redes, servidores e formação técnica pessoal e independente

Adriel Vinícius de almeida pereira
03/02/2026

Sistemas Livres: Guia Prático de Infraestrutura Linux com Debian e OpenWRT

Um laboratório pessoal de redes, servidores e formação técnica pessoal e independente

Adriel Vinícius de almeida pereira
03/02/2026

SUMÁRIO

Sumário

1. INTRODUÇÃO.....	4
1.1. Objetivo do laboratório	4
1.2. Infraestrutura Física Utilizada	5
1.2.1. Servidor AW-NU103	5
1.2.2. Roteador WR740N com OpenWRT	5
1.2.3. Cabeamento de rede	6
2. Configuração do Servidor	7
2.1. Sistema Operacional Debian Linux.....	7
2.2. Instalação do Debian em 7 etapas	7
2.2.1. Conceitos Fundamentais: SSH, apt, root, rede, DHCP	9
2.2.2. O que é SSH? (Protocolo Secure Shell).....	9
2.2.3. Administração do sistema: Root , sudo, su e PoLP	9
2.2.4. Gerenciamento de Pacotes: APT	10
2.3. Configurações Iniciais Após Instalar o Debian.....	11
3. Configuração de Rede	12
3.1.1. Topologia de Rede	15
3.1.2. Diagrama visual.....	15
3.1.3. Explicação de sub-redes, DHCP,NAT, firewall	15
3.2. Endurecimento Inicial de Segurança	15
3.3. Desktop remoto XRDP e ambiente gráfico	15
4. Problemas e soluções.....	15
4.1. Erros encontrados, como resolver	15
4.2. Lock de pacotes, Interfaces, autenticação	15
4.3. Stack de Aplicações	16
4.4. Arquitetura Final do Servidor	16
5. Serviços e Testes	Erro! Indicador não definido.
5.1. Servidor web/API/bots.....	Erro! Indicador não definido.
5.2. Laboratório de rede	Erro! Indicador não definido.
6. Possíveis melhorias	17
7. Conclusão.....	18
7.1. O que foi aprendido.....	18
8. Referências	21

1. INTRODUÇÃO

1.1. Objetivo do laboratório

O laboratório, assim como este guia, tem como objetivo aprimorar e maximizar o aprendizado em sistemas Linux, infraestrutura, redes e segurança, entre outros conceitos fundamentais. Sua estrutura e topologia controlada permitem a experimentação prática e o desenvolvimento de autonomia técnica.

A criação estruturada deste laboratório concede ao usuário a capacidade de aplicar conceitos de redes na prática e compreender como servidores realmente funcionam fora do campo puramente teórico.

Ao longo do projeto, segurança e controle serão tópicos abordados com clareza e dentro do nível necessário de detalhamento. Entender como controlar acessos e permissões, configurar firewall e SSH de forma segura, bem como compreender os riscos reais da exposição à internet, serão pontos essenciais para o aprendizado.

Com o projeto finalizado, será possível executar bots 24/7, manter e testar repositórios privados internos, hospedar projetos próprios e reduzir drasticamente a dependência de serviços pagos, proporcionando maior independência criativa e controle sobre os próprios sistemas.

Além de representar um diferencial competitivo em relação à maioria, o desenvolvimento deste projeto contribui para a construção de uma mentalidade mais madura como programador e criador, lidando com problemas reais e buscando resolvê-los da forma mais eficiente possível, sempre com clareza de propósito e direção.

Dentro deste ambiente, erros são bem-vindos, pois é por meio deles que ocorre o verdadeiro desenvolvimento técnico. Cada falha se torna uma oportunidade de aprendizado e evolução.

Mais do que apenas executar projetos, o laboratório permite compreender o funcionamento interno de sistemas, pacotes, infraestrutura e arquitetura, indo além do uso superficial das ferramentas.

Se questionado sobre o motivo deste projeto, a resposta honesta seria: maximizar meu aprendizado e desafiar-me a entender e construir um sistema relativamente complexo desde a base. Mesmo diante da complexidade de determinados procedimentos, o objetivo é concluir um projeto sólido, documentado e que possa servir como referência futura, tanto para mim quanto para outros estudantes interessados em infraestrutura e sistemas.

1.2. Infraestrutura Física Utilizada

1.2.1. Servidor AW-NU103

O servidor foi implementado utilizando o equipamento AW-NU103 (Mini PC), com as seguintes especificações:

- Processador: Intel Celeron 647 @ 1.10 GHz
- Memória RAM: 4 GB
- Armazenamento: 320 GB (HD)
- Arquitetura: 64 bits

O processador Intel Celeron 647 é um modelo de baixo consumo de energia e desempenho moderado, adequado para tarefas leves e ambientes de estudo. Por se tratar de um hardware com recursos limitados, optou-se pela instalação de uma versão mais simples do sistema operacional, priorizando a eficiência e o baixo consumo de recursos.

O armazenamento de 320 GB é suficiente para hospedar sistemas web próprios, bancos de dados leves e backups locais, atendendo adequadamente ao escopo do projeto.

Seu baixo consumo energético o torna mais econômico e viável quando comparado a desktops convencionais, sendo adequado para uso contínuo como servidor doméstico ou laboratório pessoal.

A arquitetura 64 bits garante melhor compatibilidade com softwares modernos e um gerenciamento mais eficiente da memória pelo sistema operacional.

A escolha de um hardware simples reforça a proposta do projeto: demonstrar que é possível criar um ambiente funcional e estável mesmo com recursos limitados, mantendo eficiência e organização da infraestrutura.

1.2.2. Roteador WR740N com OpenWRT

O ambiente de rede do projeto foi estruturado utilizando o roteador TP-Link TL-WR740N, configurado com o sistema operacional OpenWrt.

Especificações do equipamento

- Modelo: TP-Link TL-WR740N
- Padrão Wi-Fi: 802.11n
- Portas LAN: 4x Ethernet
- Porta WAN: 1x Ethernet

O modelo WR740N é um roteador doméstico de entrada, com recursos limitados quando comparado a equipamentos corporativos. No entanto, sua compatibilidade com o OpenWRT permite ampliar significativamente suas funcionalidades.

O OpenWRT é uma distribuição Linux embarcada voltada para dispositivos de rede. Sua utilização será justificada no decorrer do guia.

1.2.3. Cabeamento de rede

A interligação entre o servidor e o roteador foi realizada por meio de cabo de rede Ethernet categoria 6 (Cat6), com conectores padrão RJ-45.

Cabos da categoria 6 suportam maiores taxas de transferência de dados e apresentam melhor desempenho na redução de interferências eletromagnéticas quando comparados a categorias inferiores.

A utilização de conexão cabeada foi escolhida por maior estabilidade na transmissão de dados, redução de latência, menor perda de pacotes e maior confiabilidade para serviços contínuos.

Para ambientes de servidor, recomenda-se sempre o uso de conexão cabeada em vez de Wi-Fi, garantindo maior desempenho e previsibilidade na comunicação de rede.

1.2.4. Computador principal (máquina de administração)

O computador principal foi utilizado como estação de gerenciamento e configuração do ambiente.

Suas funções no projeto incluem:

- Criação da mídia de instalação do sistema operacional
- Acesso remoto ao servidor via SSH
- Transferência de arquivos
- Edição de configurações
- Desenvolvimento e testes locais

Essa máquina atua como ponto central de administração, permitindo controle completo do servidor sem necessidade de acesso físico constante ao equipamento.

A separação entre máquina administrativa e servidor é uma prática recomendada, pois permite maior organização, segurança e praticidade na manutenção da infraestrutura.

2. Configuração do Servidor

2.1. Sistema Operacional Debian Linux

Debian Minimal (Server)

Para a criação do nosso servidor, optei pelo sistema operacional Debian Minimal por saber que nosso hardware tem especificações abaixo da média. O Debian seria a melhor opção por ser extremamente leve e ter um consumo de recursos mínimo, funcionando perfeitamente em hardware fraco.

O Debian foi escolhido por ser uma distribuição extremamente leve e uma boa união entre leveza, estabilidade e funcionamento. Ideal para servidores, funciona perfeitamente para bots, APIs e Docker.

Além disso, possui um controle total do sistema e um suporte a longo prazo, ampla documentação, possibilitando futuras consultas e mudanças.

Versão utilizada

Mais especificamente, instalaremos a versão Debian 12 (Bookworm) – NetInstall (64-bit) sem interface gráfica. A instalação incluirá SSH Server e utilidades básicas do sistema.

O objetivo é manter o sistema enxuto, contendo apenas o fundamental, mas funcional para um servidor/laboratório de redes. Todas as tarefas podem ser realizadas via terminal e SSH, resultando no maior desempenho e eficiência do nosso servidor.

Função do Debian no projeto

A função do nosso Debian no projeto será atuar como cérebro da nossa rede, sendo responsável por: hospedagem de sistemas web próprios, execução de bots integrados, backend de aplicações, banco de dados local, execução de automações e um laboratório pessoal para estudos e experimentos controlados. O ambiente não afetará a rede residencial, permitindo experimentação segura e controlada.

Nossa stack no Debian será composta por Nginx, Node.js, Python, Docker, SSH, SQLite/Postgres leve, Cron, Git, UFW/iptables, PM2 e Supervisor. Com isso, nosso servidor terá a capacidade de executar múltiplos serviços simultaneamente, contendo dezenas, se não centenas, de utilidades, sendo interessantes, escaláveis e adaptáveis.

2.2. Instalação do Debian em 7 etapas

Itens necessários:

- 1 pendrive (no mínimo 4GB)
- 1 PC principal (Windows ou SO de sua preferência)
- Internet
- Teclado e monitor temporários para o servidor (computador secundário)

Etapas 1 – Baixar o Debian

Para o projeto utilizaremos a versão Debian 12 (Bookworm) – NetInstall (64-bit), como dito acima, que pode ser baixada no site oficial: <https://www.debian.org/distrib/>

O nome do arquivo será algo como:
debian-12.0.0-amd64-netinst.iso

Etapa 2 – Criar pendrive bootável (Windows)

Utilizaremos o software Rufus para a criação do pendrive bootável. Selecione o dispositivo (pendrive).

A seleção de boot deve estar na ISO do Debian, partição MBR, sistema de destino deve ser BIOS ou UEFI e sistema de arquivos FAT32.

Depois de configurado, basta iniciar o procedimento em “Start”.

Etapa 3 – Boot no PC secundário

Conecte o pendrive em uma das entradas USB do computador ainda desligado. Ligue o computador e entre no menu de boot, geralmente acessado através das teclas F12, ESC, F2 ou DEL.

Ao escolher o pendrive com o sistema onde está o Debian, abrirá o menu de instalação do sistema operacional.

Etapa 4 – Instalação

Agora é a parte onde escolhemos opções de configurações, algumas podendo ser preferenciais, como idioma e tipo de teclado.

Escolha: Graphical Install ou Install

Configurações:

- Idioma: Português
- País: Brasil
- Teclado: ABNT2
- Rede: automático (DHCP)
- Hostname: (nome de preferência)
- Domínio: (vazio por enquanto)
-

Em root password é necessário, por motivos de segurança, criar uma senha extremamente forte. Em usuário normal, digite seu nome e, por último, a senha do usuário normal (média a forte, de preferência).

Etapa 5 – Particionamento

Particionamento é o processo de dividir ou separar um recurso lógico ou físico. No nosso caso, será como direcionamos onde serão instalados nossos recursos e pacotes do Debian. Escolha as seguintes opções e confirme tudo ao final:

- Guided – use entire disk
- All files in one partition
- Finish partitioning and write changes

Etapa 6 – Seleção de pacotes

Aqui escolheremos os pacotes que serão instalados em nosso servidor e os que serão descartados. Tenha muita atenção nesse procedimento, pois é de grande importância. Na tela de seleção, marque somente:

- SSH server
- Standard system utilities

O restante deve ser desmarcado e descartado, pois não será útil em nosso projeto para a finalidade desejada.

Etapa 7 – Finalizar a instalação

Agora chegamos ao final da instalação e ao momento de maior satisfação. Instale, remova o pendrive e inicie um reboot.os

Resultado (esperado)

Você terá um servidor Debian ativo, sem interface gráfica, sendo utilizado somente o terminal para tudo, inclusive operações administrativas, com SSH ativo e pronto para uso, sendo, como já citado, leve, rápido e eficiente.

2.2.1. Conceitos Fundamentais: SSH, apt, root, rede, DHCP

Durante o processo de configuração do servidor, diversas siglas e termos técnicos foram utilizados. Para garantir maior compreensão, vamos explicar mais detalhadamente algumas das principais seções dos conceitos envolvidos.

É importante sempre estarmos aprendendo de verdade e não somente copiando aquilo que lemos. Ler, entender e aplicar de forma enxuta e limpa é o melhor jeito de aprender.

2.2.2. O que é SSH? (Protocolo Secure Shell)

O SSH (Protocolo Secure Shell) é um protocolo de rede que cria conexões criptografadas entre computadores, permitindo um acesso remoto seguro. Ele utiliza métodos de envio de pacotes e comandos com segurança por meio de uma rede não segura.

Ele permite que o usuário acesse e administre o servidor via terminal, tendo acesso total por meio de outra máquina remotamente, assim como gerenciar infraestrutura e transferir arquivos via TCP/IP.

O SSH usa criptografia para embaralhar os dados que trafegam entre as máquinas, de forma que quem os recebe captaria apenas algo semelhante a estática, nada útil, a não ser que sejam descriptografados, tornando o SSH um protocolo de transferência segura com baixa probabilidade de acesso de terceiros sem autorização.

O SSH opera sobre o protocolo TCP/IP, que é responsável pelo transporte confiável dos dados entre as máquinas e redes.

2.2.3. Administração do sistema: Root , sudo, su e PoLP

Root é a conta superusuário em sistemas baseados em Linux/Unix. O root dá controle total e irrestrito sobre o sistema de arquivos, permitindo modificar, instalar ou excluir qualquer arquivo.

Sudo é a abreviação de “SuperUser Do” e é o comando essencial para usuários Linux

executarem comandos administrativos de forma temporária, sem logar diretamente como superusuário (Root), sendo fundamental para garantir a segurança e integridade do sistema, restringindo as permissões de superusuário a apenas alguns comandos específicos.

Su, embora muito parecido com **Sudo**, tem suas diferenças. O **Su** troca o usuário atual pelo usuário root ou qualquer outro usuário especificado e permanece nesse usuário até que seja feita a saída manualmente por meio do comando “exit”. Enquanto o **Sudo** executa um único comando como superusuário e retorna ao usuário padrão imediatamente.

O **Princípio do Menor Privilégio** (Principle of Least Privilege – PoLP) determina que um usuário ou processo específico deve possuir apenas as permissões necessárias para executar sua função.

Ou seja, não utilizar o root para tarefas comuns, não conceder acesso total se não for necessário e não executar serviços administrativos com privilégios elevados sem necessidade, reduzindo o risco de erro humano, o impacto de ataques e danos acidentais.

2.2.4. Gerenciamento de Pacotes: APT

O APT (Advanced Package Tool) é basicamente um gerenciador de pacotes presente em várias distribuições Linux, utilizado para administrar pacotes, podendo baixá-los, apagá-los ou instalá-los em seu sistema.

Quase todas as distribuições baseadas em Debian possuem o APT disponível para uso, como **Ubuntu**, **Linux Mint** e **Zorin OS**, entre várias outras. Como estamos trabalhando diretamente com o sistema **Debian**, é importante compreender o funcionamento desse gerenciador.

Basicamente, utilizaremos o comando apt seguido de algum parâmetro adicional, como remoção de arquivos, instalação ou atualização. É importante lembrar que, para executar um comando APT, precisamos ter permissão de superusuário, portanto utilizaremos o sudo antes do comando.

Principais comandos:

- sudo apt update / → atualiza a lista de pacotes disponíveis.
- sudo apt upgrade -y / → atualiza os pacotes instalados.
- sudo apt install [nome-do-pacote] / → instala software
- sudo apt remove [nome-do-pacote] / → remove software
- sudo apt autoremove / → remove dependências não utilizadas
- sudo apt purge [nome-do-pacote] / → remove um pacote e suas configurações
- sudo apt clean / → apaga todo o cache de pacotes armazenado no computador
- sudo apt autoclean / → apaga o cache de pacotes que não são mais necessários ao sistema

Comando de pesquisa para pacotes instalados:

- sudo apt list --installed / → lista todos os pacotes instalados
- sudo apt search [nome-do-pacote] / → faz a localização de um pacote pesquisando apenas por um trecho do mesmo
- sudo apt show [nome-do-pacote] / → lista a informações sobre um pacote

2.3. Configurações Iniciais Após Instalar o Debian

Nesta parte vamos abordar os comandos essenciais para preparar um servidor Debian para uso em ambiente real. O objetivo é deixar o sistema atualizado, preparar a rede para futuros testes e projetos, aplicar segurança básica e instalar ferramentas essenciais.

2.3.1. Atualização do sistema

Logo após a instalação do sistema, devemos ter como primeira ação atualizar os pacotes do sistema.

Apt update
Apt upgrade -y

Apt update → Atualiza a lista de pacotes disponíveis nos repositórios.

Apt upgrade → Atualiza todos os pacotes instalados para suas versões mais recentes.

Esses comandos vão garantir segurança e estabilidade do servidor.

2.3.2. Configuração de acesso remoto (SSH)

Como dito antes, o SSH irá permitir a administração de nosso servidor remotamente.

Apt install openssh-server -y

Para ativar o serviço

systemctl enable ssh
systemctl start ssh

E para verificar seu funcionamento:

Systemctl status ssh

Se aparecer algo como “active: active (running)” o servidor já pode receber conexões remotas.

2.3.3. Verificar configuração de rede

Para acessar o servidor remotamente, é necessário saber o IP.

ip a

Procure algo como “inet 192.168.X.X/24”. Esse é o IP de seu servidor, anote-o para ser usado na conexão remota.

2.3.4. Testar acesso remoto (Pc principal)

No computador principal, dentro do cmd:

Ssh usuario@IP_DO_SERVIDOR

Como provavelmente será seu primeiro acesso, aparecerá “Are you sure you want to continue connecting (yes/no)?”. Digite: “yes”. Se conectar com sucesso, o acesso remoto está funcionando corretamente.

3. Roteador WR740N com OpenWRT

OpenWRT é um firmware de código aberto baseado em Linux, desenvolvido especificamente para roteadores e dispositivos embarcados.

Assim como o Windows é um sistema operacional para computadores e o Android para celulares, o OpenWRT funciona como um sistema operacional completo para roteadores. Ele substitui o firmware original do fabricante por um sistema livre, programável e altamente configurável, permitindo controle total da rede.

3.1 O que é Firmware?

Firmware é o software responsável por controlar diretamente o hardware de um dispositivo eletrônico.

Roteadores vendidos de fábrica possuem firmware proprietário, geralmente limitado e com poucas opções avançadas.

O OpenWRT substitui esse firmware por uma alternativa aberta, permitindo instalar pacotes, configurar serviços e transformar o roteador em um pequeno servidor Linux.

Riscos e cuidados importantes

A instalação do OpenWRT não é nenhum bixo de sete cabeças, porém é válido se atentar a alguns cuidados importantes antes e durante a instalação deste sistema para garantir a estabilidade e seu bom funcionamento.

Risco de bricker o roteador

O termo brick que em inglês significa "tijolo", é utilizado quando algum dispositivo eletrônico deixa de funcionar corretamente ou totalmente durante uma falha de firmware o tornando praticamente inutilizável.

Soft Brick é um dos dois principais tipos desse efeito, que é quando o roteador ainda liga porém não abre mais o painel web, não distribui internet, não responde corretamente à rede. Normalmente neste caso ainda é possível recuperar o equipamento com modo recovery ou apenas reinstalação do firmware.

Hard Brick ocorre quando o sistema é corrompido completamente ou o roteador não liga ou não responde. Neste tipo de situação as vezes pode ser necessário abrir o Hardware, usar cabo serial ou regravar memória flash manualmente. Para alguns iniciantes isto significa diretamente a perda definitiva de seu equipamento.

Perda de garantia

Ao instalar o OpenWRT, o firmware original do fabricante é substituído por um sistema alternativo, o que na maioria das vezes anula completamente a garantia do produto e seu suporte técnico deixa de ser válido. Por este motivo é recomendado que se utilize equipamentos fora da garantia ou destinados apenas para estudos e testes.

Versão de hardware compatível

Um dos erros mais comuns para iniciantes ou entusiastas comuns é ignorar que o mesmo equipamento pode possuir várias versões internas disponíveis, embora as vezes o equipamento tenha o mesmo nome comercial o hardware interno pode mudar coisas como:

memoria RAM, tamanho de memoria flash, chipset e procesador.

Cada versão exige um firmware específico, e podemos identificar a versão de nosso equipamento apenas verificando a etiqueta na parte inferior do equipamento. Após a confirmação da versão correta de firmware equivalente a o hardware podemos instalar sem problemas e evitar um dos principais causadores de “Brick”.

Queda de energia durante flash

Durante o processo de instalação será gravado o novo sistema na memória interna do roteador sobrescrevendo o firmware e suas configurações antigas setor por setor. Se ocorrer queda de energia seja de desligamento acidental ou reinício forçado o sistema pode ficar incompleto e o roteador não irá conseguir ser inicializado. Algumas recomendações importantes são a de nunca instalar durante instabilidade elétrica como durante tempestades, e evitar extensões frouxas ou tomadas ruins.

Mesmo que durante a instalação pareça travado aguarde pelo menos de 5 a 10 minutos para que seja feito o reinício forçado de seu equipamento, evitando problemas futuros.

Durante a instalação tenha a garantia de energia estável, e não feche o navegador durante o processo, Não desligar o roteador e nem atualizar a página, Não remova os cabos e não tente reiniciar manualmente

Embora como dito antes, o processo de instalação do OpenWRT seja relativamente algo simples, é necessário a atenção e compreensão básica do procedimento, seguindo de forma cautelosa os cuidados descritos, os riscos de falhas é mínimo e ao fim do procedimento o usuário passa a ter controle total sobre o equipamento o transformando em uma poderosa ferramenta de aprendizado e gerenciamento de redes.

3.2 O que é possível fazer com OpenWRT.

- 3.2.1 Servidor DHCP personalizado**
- 3.2.2 Firewall avançado**
- 3.2.3 VPN(wireguard/ openvpn)**
- 3.2.4 Servidor DNS LOCAL**
- 3.2.5 Bloqueio de anúncios**
- 3.2.6 VLANs**
- 3.2.7 Servidor SSH**

3.3 Conceitos Fundamentais de redes e OpenWRT

- 3.3.1 Sistema Operacional Embarcado (Embedded Linux)**
- 3.3.2 Sistema de Arquivos (Filesystem Overlay)**
- 3.3.3 Gerenciador de pacotes (opkg)**
- 3.3.4 Unified Configuration Interface**
- 3.3.5 Acesso via SSH (Administração Real)**
- 3.3.6 Imagens Firmware (Factory vs Sysupgrade)**
- 3.3.7 Reset e Recovery**
- 3.3.8 Filosofia do OpenWRT**

3.4 Guia básico de instalação

3.5 Configurações iniciais recomendadas

3.6 O que aprendi usando OpenWRT neste projeto

4. Configuração de Rede

- 4.1.1. Topologia de Rede
- 4.1.2. Diagrama visual
- 4.1.3. Explicação de sub-redes, DHCP, NAT, firewall
- 4.1.4. Endurecimento Inicial de Segurança
- 4.1.5. Desktop remoto XRDP e ambiente gráfico

- 3.6.1 (DHCP) Dynamic Host Configuration Protocol**
- 3.6.2 IP Estático**
- 3.6.3 DNS — Domain Name System**
- 3.6.4 Firewall**
- 3.6.5 WAN e LAN**
- 3.6.6 Bridge (Br-lan)**
- 3.6.7 LuCI — Interface Web**
- 3.6.8 Serviços (Daemons)**

5. Problemas e soluções

- 5.1. Erros encontrados, como resolver
- 5.2. Lock de pacotes, Interfaces, autenticação

Stack de Aplicações

Arquitetura Final do Servidor

6. Possíveis melhorias

Segurança do servidor

Apesar de o ambiente estar funcional, identifiquei que a segurança pode ser ampliada com configurações adicionais. Entre as melhorias possíveis estão a implementação de autenticação via chave SSH, desativação do login direto do usuário root e utilização de ferramentas de proteção contra tentativas de acesso indevido, como o Fail2Ban. Também seria interessante configurar um firewall para controlar portas e conexões permitidas. Essas medidas tornariam o ambiente mais próximo de um cenário real de produção.

Monitoramento do sistema

Uma melhoria importante seria a implementação de ferramentas de monitoramento contínuo do servidor. Durante o projeto, o acompanhamento foi feito manualmente, porém ferramentas específicas permitiriam visualizar consumo de CPU, memória, rede e status dos serviços em tempo real. A adoção de soluções de monitoramento ajudaria na identificação preventiva de problemas, permitindo agir antes que falhas impactem o funcionamento do sistema.

Automação de tarefas administrativas

Outra evolução natural do projeto seria automatizar tarefas repetitivas por meio de scripts em Bash. Atualizações periódicas, backups automáticos e verificações de integridade do sistema poderiam ser executadas sem intervenção manual. Isso reduziria erros humanos e aumentaria a eficiência operacional, além de aproximar o ambiente das práticas utilizadas em administração profissional de servidores.

Aplicação em ambiente real de produção

Uma melhoria futura seria aplicar todo o conhecimento adquirido em um cenário real, hospedando aplicações próprias ou projetos experimentais no servidor. Essa etapa permitiria lidar com desafios reais como disponibilidade, acesso externo, segurança contínua e manutenção a longo prazo, consolidando o aprendizado obtido durante o projeto.

7. Conclusão

7.1. O que foi aprendido

Durante a implementação e documentação do projeto, foram desenvolvidos diversos conhecimentos técnicos e operacionais, além de uma visão mais ampla de infraestrutura. Esses aprendizados serão fundamentais em minha jornada para não apenas me tornar um desenvolvedor, mas um criador de sistemas.

7.1.1. Aprendizado técnico

Funcionamento de pacotes APT e seu gerenciamento.

Apreendi como o APT realmente funciona como sistema de gerenciamento de pacotes. Compreendi a diferença entre seus comandos de forma clara, através de testes, e como cada um impacta o sistema de maneira específica. Também enfrentei problemas com travas de pacotes e dependências quebradas, o que me forçou a entender o funcionamento interno desse gerenciador, ao invés de apenas repetir comandos prontos. Esses procedimentos me deram mais segurança para administrar atualizações e instalações de forma responsável, principalmente em um ambiente de servidor.

Sistemas de permissões Linux.

Entendi como o sistema de permissões do Linux é estruturado em usuários, grupos e níveis de acesso. Compreendi a importância do acesso de “superusuário” (root) e por que seu uso deve ser restrito e estritamente bem definido nos parâmetros de segurança. Percebi que uma configuração inadequada pode comprometer completamente a integridade do sistema. Todo esse processo me fez enxergar que segurança não é algo opcional em um servidor bem estruturado, mas sim parte essencial de sua configuração básica.

Diagnóstico de erros

Durante o projeto, desenvolvi a habilidade de interpretar mensagens de erro ao invés de simplesmente ignorá-las. Apreendi a entender as saídas dos comandos e a buscar a causa raiz dos problemas. No decorrer dessa jornada, percebi que muitos erros são autoexplicativos quando analisados com paciência e atenção. Esse aprendizado foi um dos mais importantes, pois me ensinou a ser mais cauteloso diante de falhas, algo essencial para alguém que deseja viver e realmente entender o que é desenvolvimento.

Conceitos de servidor como serviço contínuo

Tive grande satisfação em entender com clareza que um servidor não é apenas uma máquina ligada, mas sim um ambiente que precisa operar de forma contínua, estável e previsível. Um servidor precisa garantir disponibilidade contínua dentro do possível, mantendo seus serviços ativos, monitorando os processos e evitando que qualquer alteração comprometa seu funcionamento. Esse entendimento mudou minha visão sobre responsabilidade técnica e me fez compreender que um servidor exige planejamento, organização e disciplina.

7.1.2. Aprendizado Operacional

Importância da documentação

Durante o desenvolvimento do projeto, percebi que a documentação não é apenas um registro do que foi feito, mas uma ferramenta essencial para a compreensão e continuidade do meu trabalho. Ao documentar cada etapa, os comandos utilizados e os problemas resolvidos, consegui organizar meu próprio raciocínio lógico e evitar cometer erros parecidos ou que já haviam sido resolvidos. A documentação reduz a dependência da nossa própria memória e permite que processos sejam reproduzidos. Como aprendi no decorrer do projeto, a memória é um recurso valioso, tanto em sistemas quanto na experiência humana.

Resolução sistemática de problemas

Outro aprendizado importante foi desenvolver uma metodologia mais estruturada para resolver problemas. Em vez de tentar soluções aleatórias e “brigar” com o terminal, passei a analisar as mensagens de erro, verificar logs, testar hipóteses sem medo e aplicar mudanças de forma controlada. Esse processo mostrou que a resolução de problemas em tecnologia depende mais de métodos corretos do que de tentativas e erros aleatórios. Aprendi a encarar minhas falhas como etapas naturais do processo de aprendizagem, e não como indicadores de incapacidade.

Erros como fonte de aprendizado

Erros são comuns, principalmente quando lidamos com algo novo e à primeira vista muito complexo, no decorrer deste projeto, tive minhas dificuldades relacionadas a pacotes, repositórios e configurações. E em vez de tratar minhas falhas como obstáculos, optei por registrá-las junto com suas soluções, com isso, percebi que os erros são uma das partes mais valiosas do aprendizado, aprofundando nosso entendimento sobre o funcionamento do sistema. Documentar meus erros transformou dificuldade momentânea em conhecimento permanente.

7.1.3. Mentalidade

Autonomia técnica

O desenvolvimento da autonomia técnica foi um dos aspectos mais importantes que identifiquei em mim ao longo desta jornada. Deixei de depender cegamente de tutoriais, passando a compreender melhor a direção das minhas ações e a confiar mais na própria investigação para resolver problemas. Desenvolvi a capacidade de avançar no projeto mesmo sem possuir todas as respostas, construindo soluções próprias e ampliando minha visão como desenvolvedor. Mais do que compreender o que estou criando, aprendi a aprender.

Organização de conhecimento

O ato de documentar e organizar meu conhecimento se tornou não só uma forma de consulta e revisão daquilo que foi feito, mas uma forma de registrar meus erros, minhas ações e minhas decisões. O método de transcrever meu conhecimento técnico se tornou uma forma de sistema pessoal que vou levar para minha carreira, entendendo que o conhecimento não precisa ser memória solta. Documentação não é registro do passado, é estrutura do aprendizado futuro.

Transformar erro em documentação

Entendi, por fim, que não precisamos demonizar nossos erros nem ocupar nossa mente o tempo inteiro tentando evitá-los a todo custo, mas sim coletá-los, entendê-los, aceitá-los e lidar com eles com maturidade, reconhecendo que farão parte de quem nos tornaremos como desenvolvedores. Documentá-los não representa uma prova futura de falta de habilidade, mas sim um registro da nossa capacidade de lidar com eles. Esta última seção é a prova de que não fui vencido pelos erros; ao contrário, eles se tornaram parte do meu projeto e parte de quem estou me tornando como profissional.

Este projeto representou mais do que a configuração de um sistema Debian; representou a consolidação de uma base sólida em administração de servidores Linux e o início de uma postura profissional voltada à responsabilidade, organização e continuidade de serviço.

8. Referências

- <https://www.debian.org/>
- <https://www.cloudflare.com>
- <https://plus.diolinux.com.br>
- <https://guialinux.uniriotec.br>